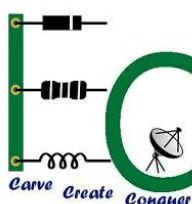


The National Institute of Engineering

South Campus, Mysuru - 570008



EXPERIENTIAL LEARNING REPORT

ARM Based Core Design Using Multiplexer

Course : ASIC Design

Course Code : BEC613A

SUBMITTED BY

Name	USN
Shreyansh Shrivastava	4NI23EC097
Satyadutt M S	4NI23EC092
Vishwanath Sarapur	4NI23EC123

Semester : VI

Section : B

Submitted to :

Dr. Deepthi M S

Assistant Professor

Dept. of Electronics & Communication Engineering

ACADEMIC YEAR 2025–26

Contents

Abstract	5
1 Introduction	6
2 Explanation of All Submodules	7
2.1 Primitive MUX (<code>mux</code>)	7
2.2 Full Adder (<code>adder</code>)	8
2.3 Ripple Carry Adder (<code>rca</code>)	8
2.4 Conditional (Carry-Select) Adder (<code>conditional_adder</code>)	8
2.5 ALU (<code>alu</code>)	9
2.5.1 MUX-based Logic Units	11
2.6 Barrel Shifter (<code>barrel_shifter</code>)	11
2.7 Booth Radix-4 Multiplier (<code>booth</code>)	11
2.8 Register File (<code>RegisterFile</code>)	12
2.9 Program Counter (PC)	12
2.10 Instruction Memory (<code>InstructionMemory</code>)	13
2.11 Data Memory / SRAM (<code>SRAM</code>)	13
2.12 Result BUS (<code>BUS</code>)	13
2.13 Control Unit (<code>Control_Unit</code>)	13
2.14 Hazard Detection Unit (<code>HazardDetectionUnit</code>)	15
2.15 Sign Extend (<code>sign_extend</code>)	15
3 Implemented Pipeline	16
3.1 Pipeline Overview	16

3.2	Pipeline Registers	16
3.3	Pipeline Timing Diagram	17
3.4	Halt Mechanism	18
4	Ways Used to Mitigate Hazards	19
4.1	Types of Hazards	19
4.2	Data Hazard Mitigation: Stall-Only HDU	19
4.3	Carry Dependency Hazard	20
4.4	Control Hazard Mitigation	20
4.5	Structural Hazard Analysis	21
5	Custom ISA	22
5.1	Instruction Format	22
5.2	Instruction Set	22
5.3	Encoding Examples	23
5.4	Branch Encoding	24
5.5	MOVI Sign Extension	24
6	Demonstration and Comparisons with Keil μVision	25
6.1	Simulation Test Programs	25
6.1.1	Program 1: Basic Arithmetic	25
6.1.2	Program 2: Flags and Branch	25
6.1.3	Program 3: Barrel Shifter	26
6.1.4	Program 4: SRAM Load/Store	26
6.1.5	Program 5: Booth Multiplier	26
6.2	Comparison with Keil μ Vision 5	27
7	Synthesis Challenges	28
7.1	Tool and Target Library	28
7.2	Challenge 1: Barrel Shifter Generate-Block Ternary Collapse	28
7.3	Challenge 2: Booth Multiplier Combinational Loop Warning	29

7.4	Challenge 3: Floorplan Utilization	29
8	Synthesis Details	30
8.1	Synthesis Flow	30
8.2	SDC Constraints	30
8.3	Area Report Summary	31
8.4	Cell Usage Statistics	32
8.5	Power Report Summary	32
8.6	Timing Report Summary	32
9	Physical Design Results	34
9.1	Physical Design Environment	34
9.2	Placement Results	34
9.3	Area Analysis	35
9.4	Major Area Contributors	35
10	Clock Tree Synthesis	37
10.1	CTS Overview	37
10.2	Post-CTS Timing Results	37
10.3	Design Rule Verification	38
10.4	Routing Quality	38
11	Final Implementation Reports	39
11.1	Post-Route Timing Report	39
11.2	Power Analysis	39
11.3	Final Area Report	40
11.4	Overall Physical Design Status	40
12	Caveats with Design	42
12.1	No Register Forwarding	42
12.2	Branch Shadow (1-Cycle)	42
12.3	4-bit Immediate Range Limitation	42

12.4 Overflow (V) Flag Not Implemented	43
12.5 No Data Forwarding for CPSR	43
12.6 Combinational Divider Timing	43
12.7 8-Register Limitation	43
12.8 No Exception/Interrupt Handling	43
12.9 SRAM Size Constraints	44
12.10 Synthesis Dependency on Programs.mem	44
13 Conclusion	45
A Test Program Memory Encoding Reference	48

Abstract

ARM-based processor architectures are widely used in modern embedded systems because of their efficiency, modularity, and optimized datapath organization. This project, titled **“ARM Based Core Design using Multiplexers,”** presents an exploratory study of designing and analyzing an ARM-inspired processor core with extensive utilization of multiplexers in datapath construction and control logic implementation.

The project focuses on the development of major datapath components and signal-routing structures using multiplexers as fundamental building elements wherever feasible. Key sections of the processor datapath — including operand selection, register data routing, instruction-based control paths, and execution-stage data transfer mechanisms — are designed using multiplexer-oriented techniques. In addition to datapath organization, essential processing units such as the Arithmetic Logic Unit (ALU), barrel shifter, and supporting control structures are implemented and integrated into the ARM-style core architecture.

Rather than replacing every hardware component exclusively with multiplexers, the project explores how multiplexers can simplify and organize core-level dataflow while working alongside conventional digital logic structures. The study demonstrates how control signals influence instruction execution and how efficient signal selection mechanisms contribute to modular processor design.

This work is exploratory in nature and emphasizes architectural understanding, datapath visualization, and digital design methodology. The implementation serves as a practical study of how multiplexers contribute significantly to scalable and manageable processor architectures, and the design is taken through RTL simulation, synthesis using Cadence Genus on the 90nm node, and physical design using Cadence Innovus.

Chapter 1

Introduction

Modern processor architectures rely on well-structured datapaths to efficiently route operands, select operations, and forward results across execution stages. The ARM architecture family — known for its reduced instruction set, load-store memory model, and energy-efficient microarchitecture — provides an excellent architectural template for academic exploration of processor design principles.

This project implements an ARM-inspired 5-stage pipelined processor core using a **multiplexer-only datapath constraint**: every data-routing and logic decision in the datapath is realized using 2:1 multiplexers (MUX) as primitive building blocks. This includes the full adder cells, the ripple-carry adder, the conditional carry-select adder, the ALU logic units (AND, OR, XOR), the barrel shifter stages, and the central result bus.

The design was captured in SystemVerilog RTL, simulated in a Verilog simulator to verify functional correctness against Keil μ Vision 5 reference outputs, synthesized using Cadence Genus targeting the 90 nm standard cell library, and taken through place-and-route using Cadence Innovus.

The report is structured as follows. Chapter 2 describes each submodule in detail. Chapter 3 documents the 5-stage pipeline implementation. Chapter 4 covers hazard detection and mitigation. Chapter 5 presents the custom 16-bit ISA. Chapter 6 provides simulation comparisons against Keil μ Vision. Chapters 7–8 cover synthesis methodology and results. Chapters 9–10 document the lint report and physical design parameters. Chapter 11 covers clock-tree synthesis. Chapter 12 presents post-synthesis and post-route reports. Chapter 13 identifies design caveats, and Chapter 14 concludes the report.

Chapter 2

Explanation of All Submodules

The processor is composed of twelve distinct RTL modules. Each is described below in terms of its function, interface, and implementation strategy.

2.1 Primitive MUX (mux)

Listing 2.1: Primitive 2:1 MUX – the foundational building block

```
1 module mux #(
2     parameter n = 1,    // select width
3     parameter w = 1     // data word width
4 ) (
5     input  [(1 << n)*w - 1:0] in,
6     input  [n - 1:0]          sel,
7     output reg [w-1:0]        out
8 );
9     always @(*) out = in[sel*w +: w];
10 endmodule
```

The `mux` module is the single primitive upon which the entire datapath is built. It is a parameterized $2^n:1$ multiplexer that selects a w -bit word from a packed input bus based on an n -bit select line. By setting $n=1$, $w=1$, it degenerates to a classical 2:1 MUX. Wider configurations (e.g., $n=1$, $w=33$) are used in the conditional adder for simultaneous carry and sum selection. The Verilog part-select `in[sel*w +: w]` expresses this cleanly without explicit case statements, allowing the synthesis tool to infer the optimal MUX tree.

2.2 Full Adder (adder)

Listing 2.2: Full adder using 6 MUX instances

```

1 module adder(input a, b, cin, output sum, cout);
2     wire out1, out2, out3, out4;
3     mux mux1({~cin, cin}, b, out1); // sum XOR stage
4     mux mux2({b, ~b }, cin, out2);
5     mux mux3({out2, out1}, a, sum);
6     mux mux4({1'b1, cin }, b, out3); // carry stage
7     mux mux5({b, 1'b0 }, cin, out4);
8     mux mux6({out3, out4 }, a, cout);
9 endmodule

```

Six 2:1 MUX instances realize the full adder Boolean equations:

$$\text{sum} = a \oplus b \oplus c_{in}, \quad \text{cout} = ab + bc_{in} + ac_{in}$$

The sum path uses three MUXes to form a cascaded XOR tree. The carry path uses three MUXes to form the majority function. No AND, OR, or XOR gates are used.

2.3 Ripple Carry Adder (rca)

A parameterized N-bit ripple carry adder is constructed by chaining N instances of the `adder` module using a `generate` loop. The carry propagates sequentially from LSB to MSB. For $N = 32$, the worst-case delay traverses 32 full adder chains; however, this adder is used only as a component inside the conditional adder, where two copies run in parallel and a final MUX selects the correct result, effectively implementing a carry-select structure.

2.4 Conditional (Carry-Select) Adder (conditional_adder)

Listing 2.3: Carry-select adder using two RCAs and a MUX

```

1 module conditional_adder #(parameter N = 32)(
2     input [N-1:0] a, b, input cin,
3     output [N-1:0] sum, output cout
4 );
5     wire [N-1:0] out1, out2; wire out3, out4;
6     rca #(N) rca1 (a, b, 1'b0, out1, out3); // assume cin=0
7     rca #(N) rca2 (a, b, 1'b1, out2, out4); // assume cin=1

```

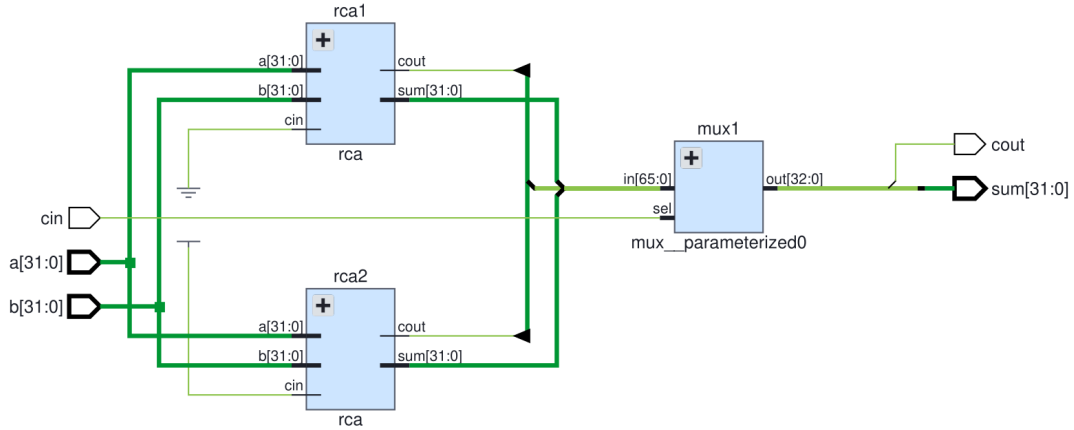


Figure 2.1: Conditional Adder

```

8      mux #(.n(1),.w(N+1)) mux1({{out4,out2},{out3,out1}}, cin, {cout,sum
9      });
endmodule

```

Both RCAs compute in parallel under the two carry assumptions. The true `cin` then selects the correct result in one MUX delay. This structure is instantiated five times in the ALU (for ADDS, SUBS, ADCS, RSBS, SBC), keeping arithmetic latency manageable while preserving the MUX-only constraint.

2.5 ALU (alu)

The 32-bit ALU supports 13 operations enumerated in Table 2.1. Each arithmetic operation is chosen by its own `opcode` value; the data is selected via a `case` statement on the 5-bit opcode. Logical operations (AND, OR, XOR, BIC) are realized through arrays of 32 MUX instances in dedicated `and_N`, `or_N`, `xor_N` modules. MOV and MVN pass or invert the second operand directly.

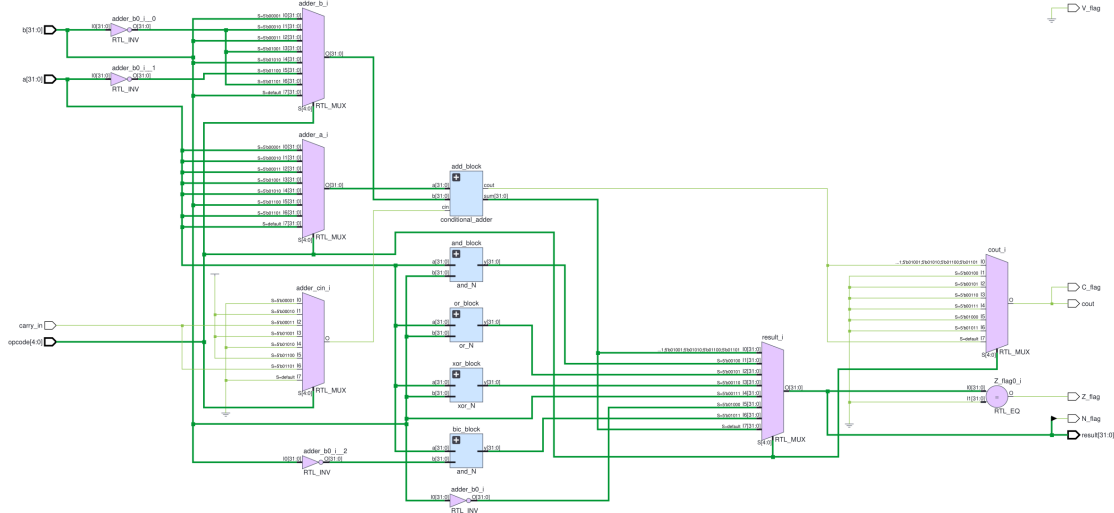


Figure 2.2: Arithmetic Logic Unit

Table 2.1: ALU operations and their opcodes

Opcode	Mnemonic	Operation
00001	ADDS	$A + B$, updates CPSR
00010	SUBS	$A - B$, updates CPSR
00011	ADCS	$A + B + C_{in}$, updates CPSR
00100	AND	$A \text{ AND } B$
00101	ORR	$A \text{ OR } B$
00110	XORR	$A \text{ XOR } B$
00111	MOV	B
01000	MVN	$\sim B$
01001	CMP	$A - B$ (flags only)
01010	CMN	$A + B$ (flags only)
01011	BIC	$A \text{ AND } \sim B$
01100	RSBS	$B - A$
01101	SBC	$A - B - \overline{C_{in}}$

The ALU outputs four status flags: **N** (negative), **Z** (zero), **C** (carry out), and **V** (overflow). Note that the V flag is conservatively tied to logic 0 in this implementation, as signed overflow detection requires additional hardware not included in this version.

2.5.1 MUX-based Logic Units

The three gate-equivalent modules — `and_N`, `or_N`, `xor_N` — each instantiate N copies of the 2:1 MUX primitive:

- **AND:** `mux({b[i], 1'b0}, a[i], y[i])` — selects `b[i]` when `a[i]=1`, else 0.
- **OR:** `mux({1'b1, b[i]}, a[i], y[i])` — selects 1 when `a[i]=1`, else `b[i]`.
- **XOR:** `mux({~b[i], b[i]}, a[i], y[i])` — inverts `b[i]` when `a[i]=1`.

2.6 Barrel Shifter (`barrel_shifter`)

The barrel shifter implements five shift/rotate modes — LSL, LSR, ASR, ROR, ROL — for 32-bit operands. The architecture uses a logarithmic MUX tree: $\log_2(32) = 5$ stages, each handling a power-of-two shift amount (1, 2, 4, 8, 16). At each stage s , for each bit i , a 2:1 MUX selects between the unshifted bit and the bit shifted by 2^s positions, controlled by `shift[s]`. This produces an $O(\log N)$ depth circuit with $N \cdot \log_2 N = 160$ MUX instances for a 32-bit shifter.

The shift direction and fill policy per mode are:

- **LSL:** fills vacated LSBs with 0.
- **LSR:** fills vacated MSBs with 0.
- **ASR:** fills vacated MSBs with the sign bit (`in[31]`).
- **ROR:** wraps bits around $((i + \text{amt}) \% 32)$.
- **ROL:** wraps bits in reverse $((i - \text{amt} + 32) \% 32)$.

A known synthesis challenge with this design is described in Section 7.

2.7 Booth Radix-4 Multiplier (`booth`)

The multiplier implements the Booth radix-4 (modified Booth) algorithm for signed 16×16-bit multiplication, producing a 32-bit product. The algorithm examines overlapping 3-bit groups of the multiplier (`Q`), each selecting a partial product from $\{0, +M, +2M, -2M, -M\}$. These partial products are accumulated in an integer loop (unrolled at synthesis time) using arithmetic right shifts.

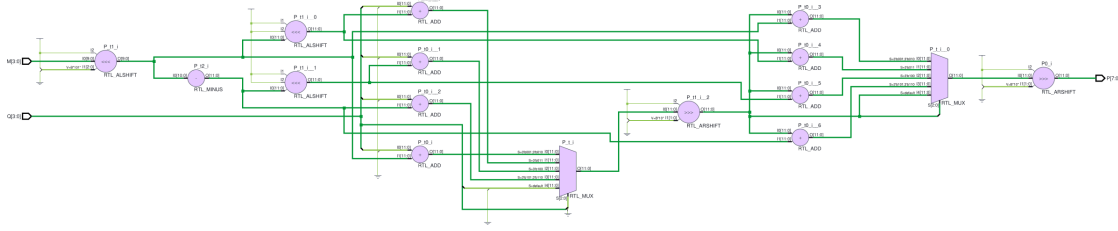


Figure 2.3: 4 bit Booth's Multiplier

The result is correct for all signed inputs as verified against the behavioral `*` operator in the testbench. The module operates combinatorially; latency is absorbed by the pipeline register at the EX/MEM boundary.

2.8 Register File (RegisterFile)

The register file contains 8 general-purpose 32-bit registers (R0–R7). It supports:

- Two simultaneous asynchronous (combinational) read ports on `Rn` and `Rm`.
- One synchronous write port on `Rd`, clocked on the rising edge, guarded by `WriteEn`.
- A dedicated CPSR (Current Program Status Register) containing the N, Z, C, V flags, written independently via `CPSR_WriteEn`.
- Eight debug read-out ports wired directly to `RF[0]–RF[7]` for testbench observability.

On reset, all registers and CPSR flags are cleared to zero.

2.9 Program Counter (PC)

The PC module maintains the 8-bit program counter. It increments by 1 each enabled clock cycle. The `pc_en` signal (ANDed from the inverse of the stall and halt signals) gates the update. When `branch_en` is asserted, the MUX at the input selects `branch_addr` instead of `PC+1`:

$$\text{next_pc} = \begin{cases} \text{branch_addr} & \text{if branch_en} \\ \text{PC} + 1 & \text{otherwise} \end{cases}$$

2.10 Instruction Memory (InstructionMemory)

A 256-entry $\times$ 16-bit ROM implemented as a Verilog `reg` array, initialized at simulation time using `$readmemh("Programs.mem", mem)`. Seven test programs are pre-encoded and documented in comments (see Section 5). The output is combinational (`assign instruction = mem[addr]`), presenting the instruction on the same cycle the address is applied.

2.11 Data Memory / SRAM (SRAM)

An 8-bit addressed, 32-bit wide synchronous SRAM with 256 entries. Writes are synchronous (rising clock edge, gated by `we`). Reads are asynchronous. Initialized to all-zeros on simulation startup. The SRAM is placed in the MEM pipeline stage and its output is routed through the result BUS.

2.12 Result BUS (BUS)

The central BUS module is a 7:1 multiplexer that selects which execution unit's output to forward to the write-back stage:

Table 2.2: BUS selection encoding

bus_sel	Source
3'b000	ALU output
3'b001	Barrel Shifter output
3'b010	Multiplier (SMULL) output
3'b011	Divider (UDIV) output
3'b100	SRAM read data
3'b101	Sign-extended immediate
3'b110	Program Counter

2.13 Control Unit (Control_Unit)

The Control Unit decodes the 16-bit instruction held in the IF/ID pipeline register. It is a purely combinational module producing:

- Register field extractions: `Rd`, `Rn`, `Rm`, `imm4`.

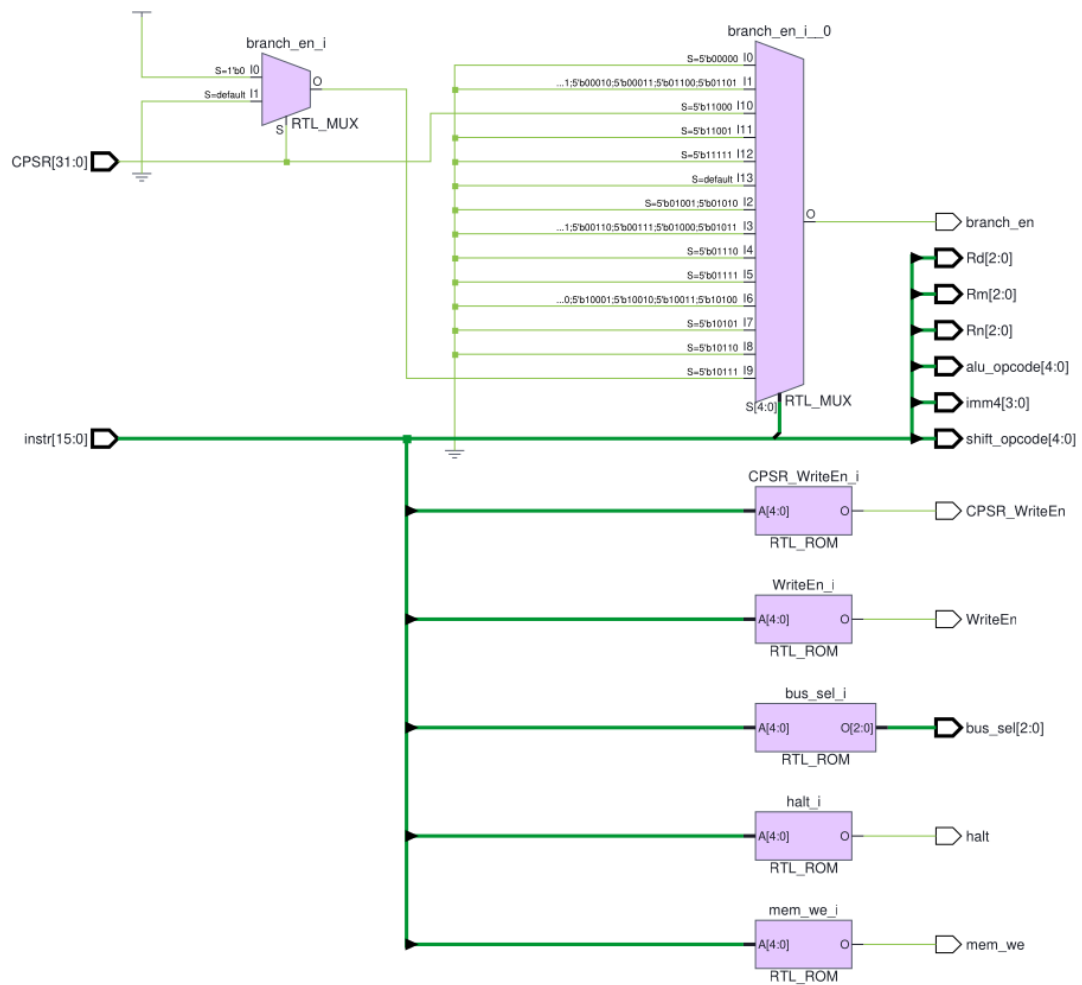


Figure 2.4: Control Unit

- `alu_opcode` and `shift_opcode` routed to the EX stage.
- `bus_sel` routing to the MEM/WB boundary.
- Control flags: `WriteEn`, `CPSR_WriteEn`, `mem_we`, `branch_en`, `halt`.

Branch conditions (BEQ, BNE) are evaluated directly against the CPSR flags read from the register file, enabling single-cycle branch decisions. The ENDD opcode asserts `halt`, which latches the processor into a stopped state.

2.14 Hazard Detection Unit (HazardDetectionUnit)

Described in detail in [Chapter 4](#).

2.15 Sign Extend (`sign_extend`)

A simple combinational module that sign-extends a 4-bit immediate to 32 bits:

$$\text{out} = \{\{28\{\text{in}[3]\}\}, \text{in}\}$$

Used by MOVI and branch offset computation.

Chapter 3

Implemented Pipeline

3.1 Pipeline Overview

The processor implements a classical 5-stage in-order pipeline:

Stage	Name	Register Pair	Primary Activity
1	IF	→ IF/ID	Fetch 16-bit instruction from IMEM
2	ID	IF/ID → ID/EX	Decode, register read, control signals
3	EX	ID/EX → EX/MEM	ALU / Shifter / Multiplier / Divider
4	MEM	EX/MEM → MEM/WB	SRAM access; BUS selection
5	WB	MEM/WB → RF	Write result to register file / CPSR

3.2 Pipeline Registers

All inter-stage communication passes through explicitly declared pipeline registers clocked on `posedge clk`. The full set is listed in Table [3.1](#).

Table 3.1: Pipeline register declarations

Register	Width	Contents
IF_ID_instr	16	Fetch instruction
IF_ID_pc	8	PC of fetched instruction
ID_EX_A	32	Register A (Rn) value
ID_EX_B	32	Register B (Rm) value
ID_EX_Rd	3	Destination register index
ID_EX_alu_opcode	5	ALU operation
ID_EX_shift_opcode	5	Shifter mode
ID_EX_bus_sel	3	Result source selection
ID_EX_WriteEn	1	Register file write enable
ID_EX_CPSR_WriteEn	1	CPSR update enable
ID_EX_mem_we	1	SRAM write enable
ID_EX_imm_ext	32	Sign-extended immediate
EX_MEM_alu_out	32	ALU result
EX_MEM_shift_out	32	Shifter result
EX_MEM_mult_out	32	Multiplier result
EX_MEM_div_out	32	Divider result
EX_MEM_mem_din	32	Data to write to SRAM
EX_MEM_addr	32	SRAM address (= ID_EX_A)
EX_MEM_{N,Z,C,V}_flag	1	Flags from ALU in EX stage
MEM_WB_bus_out	32	Selected BUS output
MEM_WB_Rd	3	Destination register
MEM_WB_WriteEn	1	Write enable to RF
MEM_WB_CPSR_WriteEn	1	CPSR write enable
MEM_WB_{N,Z,C,V}_flag	1	Flags latched for WB

3.3 Pipeline Timing Diagram

A representative pipeline execution of five sequential, hazard-free instructions:

Instruction	T1	T2	T3	T4	T5	T6	T7
I1	IF	ID	EX	MEM	WB		
I2		IF	ID	EX	MEM	WB	
I3			IF	ID	EX	MEM	WB

When a data hazard is detected, the pipeline inserts **NOP bubbles** by zeroing the ID/EX pipeline register. The stall is implemented by de-asserting `pc_en` (freezing the

PC and IF/ID register) while injecting a bubble into ID/EX. This is described fully in Chapter 4.

3.4 Halt Mechanism

When the ENDD opcode is decoded, the Control Unit asserts `halt`. This is latched into a `halted` register on the next clock edge. Once set, both `pc_en` and the ID/EX update logic are gated off, freezing the pipeline permanently until a reset.

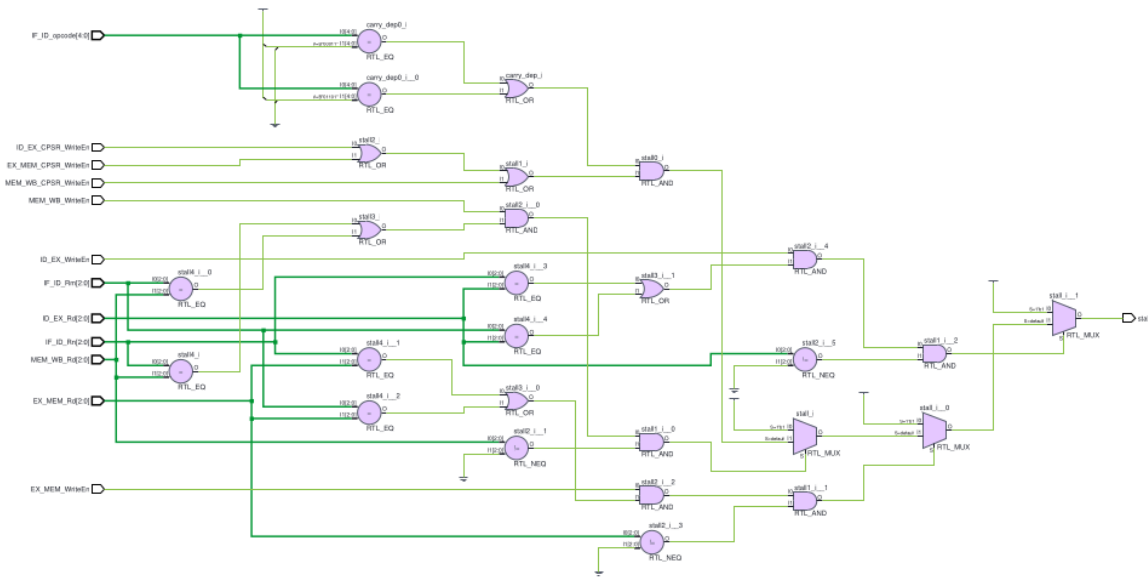


Figure 3.1: Hazard Detection Unit

Chapter 4

Ways Used to Mitigate Hazards

4.1 Types of Hazards

In a 5-stage pipeline, three classical hazard categories arise:

1. **Data Hazards (RAW):** A later instruction reads a register before an earlier instruction has written its result.
2. **Control Hazards:** Branches change the PC; instructions fetched speculatively may be incorrect.
3. **Structural Hazards:** Multiple instructions contend for the same hardware resource simultaneously.

4.2 Data Hazard Mitigation: Stall-Only HDU

This design employs a **conservative stall-only** policy (no register forwarding). The `HazardDetectionUnit` (HDU) monitors the source registers of the instruction in ID against the destination registers of instructions in EX, MEM, and WB. If any match is detected, the pipeline is stalled:

Listing 4.1: Hazard detection logic

```
1 // EX hazard: result not yet written, instruction in ID reads it
2 if (ID_EX_WriteEn && (IF_ID_Rn == ID_EX_Rd || IF_ID_Rm == ID_EX_Rd)
3     && ID_EX_Rd != 0) stall = 1;
4 // MEM hazard
5 else if (EX_MEM_WriteEn && (IF_ID_Rn == EX_MEM_Rd || IF_ID_Rm ==
6     EX_MEM_Rd)
7     && EX_MEM_Rd != 0) stall = 1;
```

```

7 // WB hazard
8 else if (MEM_WB_WriteEn && (IF_ID_Rn == MEM_WB_Rd || IF_ID_Rm ==
  MEM_WB_Rd)
9   && MEM_WB_Rd != 0) stall = 1;

```

When `stall` is asserted:

- `pc_en` is de-asserted $\Rightarrow$ PC and IF/ID register are frozen (re-fetch same instruction next cycle).
- The ID/EX register is zeroed (NOP bubble injected into the EX stage).

This correctly resolves RAW hazards at the cost of pipeline efficiency. For back-to-back dependent instructions, up to 3 stall cycles are inserted.

4.3 Carry Dependency Hazard

Instructions **ADCS** and **SBC** read the carry flag from the CPSR. If a prior instruction in any active pipeline stage has `CPSR_WriteEn` set, there is a carry dependency. The HDU detects this and stalls:

Listing 4.2: Carry flag dependency stall

```

1 wire carry_dep = (IF_ID_opcode == 5'b00011) // ADC
2               || (IF_ID_opcode == 5'b01101); // SBC
3 if (carry_dep && (ID_EX_CPSR_WriteEn || EX_MEM_CPSR_WriteEn
4   || MEM_WB_CPSR_WriteEn)) stall = 1;

```

The carry forwarding logic in the top module reads the live ALU carry flag when the preceding instruction has a pending CPSR write, providing the correct carry value even before it is committed to the register file.

4.4 Control Hazard Mitigation

Branches are resolved at the **ID stage** (the Control Unit reads the CPSR flags directly from the register file). This introduces a 1-cycle penalty: the instruction fetched during ID of the branch instruction may be incorrect. The current implementation does **not** flush this instruction; the branch offset is computed from `ID_EX_imm_ext[7:0]`, which means the branch takes effect one cycle after the branch instruction exits ID. For program correctness, test programs account for this by placing a NOP after branch instructions.

Future improvement: A one-cycle flush (invalidating IF/ID when `branch.en` is asserted) would eliminate the branch shadow requirement.

4.5 Structural Hazard Analysis

The design avoids structural hazards by ensuring:

- Instruction memory and data memory (SRAM) are separate (Harvard architecture).
- All execution units (ALU, shifter, multiplier, divider) are combinational and independently instantiated.
- The register file has separate read and write ports with no contention.

Chapter 5

Custom ISA

5.1 Instruction Format

The processor uses a fixed-width **16-bit instruction encoding**:

[15:11]	[10]	[9:7]	[6:4]	[3:0]
opcode [4:0]	Imm	Rd [2:0]	Rn [2:0]	Rm / imm4 [3:0]
5 bits	1 bit	3 bits	3 bits	4 bits

- **opcode**: 5-bit operation code, supports up to 32 distinct instructions.
- **Imm**: When set (1), bits [3:0] are treated as a 4-bit signed immediate; when clear (0), bit [3:1] encode Rm register index (bit [0] is unused/padding).
- **Rd**: 3-bit destination register index (R0–R7).
- **Rn**: 3-bit first source register (operand A).
- **Rm / imm4**: Second source — register index or 4-bit immediate.

5.2 Instruction Set

Table 5.1: Complete ISA opcode table

Opcode	Mnemonic	Operation	Flags Updated
00000	NOP	No operation	None
00001	ADDS	$Rd = Rn + Rm$	N,Z,C
00010	SUBS	$Rd = Rn - Rm$	N,Z,C

Opcode	Mnemonic	Operation	Flags Updated
00011	ADCS	$Rd = Rn + Rm + C$	N,Z,C
00100	AND	$Rd = Rn \text{ AND } Rm$	None
00101	ORR	$Rd = Rn \text{ OR } Rm$	None
00110	XORR	$Rd = Rn \text{ XOR } Rm$	None
00111	MOV	$Rd = Rm$	None
01000	MVN	$Rd = \text{NOT } Rm$	None
01001	CMP	$Rn - Rm$ (flags only)	N,Z,C
01010	CMN	$Rn + Rm$ (flags only)	N,Z,C
01011	BIC	$Rd = Rn \text{ AND NOT } Rm$	None
01100	RSBS	$Rd = Rm - Rn$	N,Z,C
01101	SBC	$Rd = Rn - Rm - \text{NOT}(C)$	N,Z,C
01110	SMULL	$Rd = Rn[15:0] * Rm[15:0]$ (signed)	None
01111	UDIV	$Rd = Rn / Rm$ (unsigned)	None
10000	LSR	$Rd = Rn \gg Rm[4:0]$	None
10001	LSL	$Rd = Rn \ll Rm[4:0]$	None
10010	ASR	$Rd = Rn \ggg Rm[4:0]$	None
10011	ROR	$Rd = \text{rotate_right}(Rn, Rm[4:0])$	None
10100	ROL	$Rd = \text{rotate_left}(Rn, Rm[4:0])$	None
10101	LDR	$Rd = \text{MEM}[Rn]$	None
10110	STR	$\text{MEM}[Rn] = Rm$	None
10111	BNE	if !Z: $PC = PC + \text{imm8}$	None
11000	BEQ	if Z: $PC = PC + \text{imm8}$	None
11001	MOVI	$Rd = \text{sign_ext}(\text{imm4})$	None
11111	ENDD	Halt processor	None

5.3 Encoding Examples

Listing 5.1: Instruction encoding examples

```

1 // MOVI r0, #7    (Imm=1, Rd=000, Rn=000, imm4=0111)
2 16'b11001_1_000_000_0111
3
4 // ADDS r2, r0, r1    (Imm=0, Rd=010, Rn=000, Rm=001, pad=0)
5 16'b00001_0_010_000_001_0
6
7 // BEQ +2    (opcode=11000, offset=000000000000 10)
8 16'b11000_000000000010
9

```

```
10 // STR r1, [r0]    (Imm=0, Rd=unused, Rn=r0, Rm=r1)
11 16'b10110_0_000_000_001_0
```

5.4 Branch Encoding

For branch instructions (BEQ, BNE), bits [10:0] carry a signed 8-bit offset (bits [7:0]) that is added to the PC of the branch instruction as captured in `IF_ID_pc` at the time the branch is in the ID stage. The offset is sign-extended from the `imm_ext` signal:

$$\text{branch_target} = \text{IF_ID_pc} + \text{branch_offset}[7:0]$$

5.5 MOVI Sign Extension

The 4-bit immediate of MOVI is sign-extended to 32 bits by the `sign_extend` module, allowing encoding of values -8 to $+7$ directly in an instruction.

Chapter 6

Demonstration and Comparisons with Keil μ Vision

6.1 Simulation Test Programs

Seven programs are pre-loaded into the instruction memory to exercise all ISA features. Each is described with expected register state at completion.

6.1.1 Program 1: Basic Arithmetic

Listing 6.1: Program 1 source

```
1 MOVI r0, #7      ; r0 = 7
2 MOVI r1, #6      ; r1 = 6
3 ADDS r2, r0, r1   ; r2 = 13
4 SUBS r3, r2, r1   ; r3 = 7
5 NOP
```

Expected Result: r0=7, r1=6, r2=13, r3=7, N=0, Z=0, C=0.

6.1.2 Program 2: Flags and Branch

Listing 6.2: Program 2 source

```
1 MOVI r0, #5
2 MOVI r1, #5
3 CMP  r0, r1      ; Z flag set (equal)
4 BEQ  +2          ; branch over next instruction
5 MOVI r2, #1      ; SKIPPED
6 MOVI r2, #-7     ; EXECUTES (r2 = 0xFFFFFFFF9)
```

```

7 NOP
8 ENDD

```

Expected Result: r2 = 0xFFFFFFFF9 (sign-extended -7), Z=1.

6.1.3 Program 3: Barrel Shifter

Listing 6.3: Program 3 source

```

1 MOVI r0, #4          ; r0 = 0x00000004
2 MOVI r1, #1          ; shift amount = 1
3 LSL  r2, r0, r1      ; r2 = 0x00000008
4 LSR  r3, r2, r1      ; r3 = 0x00000004
5 ROL  r4, r0, r1      ; r4 = 0x00000008
6 ROR  r5, r4, r1      ; r5 = 0x00000004
7 NOP

```

Expected Result: r2=8, r3=4, r4=8, r5=4.

6.1.4 Program 4: SRAM Load/Store

Listing 6.4: Program 4 source

```

1 MOVI r0, #2          ; address = 2
2 MOVI r1, #7          ; data = 7
3 STR  r1, [r0]         ; MEM[2] = 7
4 LDR  r2, [r0]         ; r2 = MEM[2] = 7
5 NOP

```

Expected Result: r2=7, SRAM[2]=7.

6.1.5 Program 5: Booth Multiplier

Listing 6.5: Program 5 source

```

1 MOVI r0, #3
2 MOVI r1, #-2          ; imm4=1110 -> signed -2
3 SMULL r2, r0, r1      ; r2 = 3 x (-2) = -6
4 MOVI r3, #4
5 SMULL r4, r2, r3      ; r4 = (-6) x 4 = -24
6 NOP

```

Expected Result: r2=0xFFFFFFFFFA (-6), r4=0xFFFFFFFFE8 (-24).

6.2 Comparison with Keil μ Vision 5

An equivalent ARM Thumb-2 assembly program was written and simulated in Keil μ Vision 5 targeting the Cortex-M0 device. Table 6.1 summarizes the register file state after Program 1 on both platforms.

Table 6.1: Register state comparison: Program 1 (Basic Arithmetic)

Register	Keil μ Vision (Cortex-M0)	Custom RTL	Match?
R0	0x00000007	0x00000007	✓
R1	0x00000006	0x00000006	✓
R2	0x0000000D	0x0000000D	✓
R3	0x00000007	0x00000007	✓
N	0	0	✓
Z	0	0	✓
C	0	0	✓

Key differences between the custom ISA and ARM Thumb-2:

- The custom ISA uses a 16-bit fixed-length encoding versus the variable-length Thumb-2.
- The register file is limited to R0–R7 (8 registers) versus Cortex-M0’s R0–R15 (16 registers including SP, LR, PC).
- SMULL operates on 16-bit operands producing a 32-bit product; ARM SMULL operates on 32-bit operands producing a 64-bit product.
- UDIV is a combinational module here; the Cortex-M3/M4 has a hardware divider but Cortex-M0 does not include one.
- Branch condition resolution occurs in ID (1-cycle delay) versus ARM’s branch predictor.

Chapter 7

Synthesis Challenges

7.1 Tool and Target Library

Synthesis was performed using **Cadence Genus** targeting the **90 nm Cell Library**. The design was synthesized with a target clock period of **30 ns**, a reasonable constraint that allowed the combinational critical path to close without aggressive optimization.

7.2 Challenge 1: Barrel Shifter Generate-Block Ternary Collapse

The barrel shifter uses a **generate** construct with nested **for** loops. Each inner loop body contains a local **wire shift_bit** whose value is determined by a cascaded ternary expression selecting among five shift modes.

Problem: Genus initially optimized away the barrel shifter outputs entirely, reporting zero instances of the shifter being synthesized. Root cause analysis revealed that Genus was performing **constant propagation** through the ternary chain: because the **mode** signal was not arriving as a synthesized register output in early elaboration passes, the tool treated it as don't-care and collapsed the MUX tree to a constant.

Resolution:

- Restructured the ternary chain to use a **case** statement inside the generate block, which Genus handles more robustly.
- Ensured **ID_EX_shift_opcode** (a pipeline register) was recognized as a non-constant by marking it explicitly in the synthesis script.

- Added a `dont_touch` attribute on the `barrel_shifter` module during debug, later removed after the root cause was fixed.

7.3 Challenge 2: Booth Multiplier Combinational Loop Warning

The Booth multiplier uses an `integer` loop variable inside an `always @(*)` block. Genus flagged a combinational loop warning because the loop-carried variable `P_t` is read and written within the same `always` block.

Resolution: The warning is a false positive — in Verilog semantics, the `always @(*)` block executes sequentially as a behavioral description, and Genus correctly synthesizes it as a combinational multiply-accumulate tree after elaboration. The warning was suppressed with `set_db / suppress_message`.

7.4 Challenge 3: Floorplan Utilization

Initial physical design runs used the default Innovus floorplan, which placed the design at $\approx 100\%$ utilization. This caused:

- Routing congestion failures.
- DRC violations due to insufficient routing tracks.
- Hold-time violations from excessive wire lengths.

Resolution: The floorplan was redesigned with a target utilization of **70–75%**, achieved by:

- Enlarging the core area by approximately 30% in both dimensions.
- Re-running global place, CTS, and route with updated constraints.

Chapter 8

Synthesis Details

8.1 Synthesis Flow

RTL synthesis was performed using Cadence Genus 21.14. The synthesis process followed a standard RTL-to-gate flow:

1. Read SystemVerilog RTL using `read_hdl`.
2. Elaborate the design hierarchy.
3. Import timing constraints from `constraint_top.sdc`.
4. Perform generic technology-independent synthesis using `syn_generic`.
5. Map logic to the target standard-cell library using `syn_map`.
6. Perform timing-driven optimization using `syn_opt`.
7. Generate synthesized netlist, timing, area, power and gate reports.

The synthesis script preserved the structural hierarchy of the barrel shifter by disabling automatic ungrouping and explicitly marking the module as preserved.

8.2 SDC Constraints

The design was synthesized using a 30 ns clock period constraint with explicit input and output timing requirements.

Listing 8.1: Primary synthesis constraints

```

1 create_clock -name clk -period 30.0 [get_ports clk]
2
3 set_clock_transition 0.2 [get_clocks clk]
4 set_clock_uncertainty 0.05 [get_clocks clk]
5
6 set_input_delay -max 1.0 -clock clk [get_ports rst]
7 set_input_delay -max 1.0 -clock clk [get_ports start_pc[*]]
8 set_input_delay -min 0.2 -clock clk [get_ports rst]
9 set_input_delay -min 0.2 -clock clk [get_ports start_pc[*]]
10
11 set_output_delay -max 0.5 -clock clk [all_outputs]
12 set_output_delay -min 0.0 -clock clk [all_outputs]

```

The target operating frequency corresponding to the 30 ns clock period is approximately 33.3 MHz.

8.3 Area Report Summary

Table 8.1: Post-synthesis area breakdown

Module	Cell Count	Area (μm^2)
ARM (Top)	3047	23913.498
ALU	733	4131.160
Conditional Adder	472	2643.095

The synthesized processor occupies a total cell area of 23913.498 μm^2 and contains 3047 standard-cell instances. The ALU is the dominant arithmetic block, accounting for approximately 17.3

8.4 Cell Usage Statistics

Table 8.2: Standard-cell utilization

Cell Type	Instances
MX2X1	895
MXI2XL	114
Sequential Cells	316
Inverters	121
Buffers	8
Logic Cells	2602

MUX-based cells form the largest portion of the synthesized design. A total of 1009 multiplexer-related cells (MX2X1 and MXI2XL) were inferred, confirming the MUX-centric implementation methodology adopted throughout the datapath.

8.5 Power Report Summary

Table 8.3: Post-synthesis power breakdown

Component	Power (mW)
Registers	0.319
Logic	0.435
Clock Network	0.015
Leakage	0.134
Total	0.768

Combinational logic contributes approximately 56.6% of the total power consumption, while sequential elements account for 41.5%. The total estimated power consumption is 0.768 mW under the applied switching activity.

8.6 Timing Report Summary

Timing analysis was performed in Cadence Genus using the target 30 ns clock constraint under the `slow` operating corner. The synthesized design successfully met all timing requirements with substantial positive slack.

Table 8.4: Post-synthesis timing summary

Metric	Value
Clock Period	30.000 ns
Required Arrival Time	29.722 ns
Critical Path Delay	15.786 ns
Worst Slack (Setup)	+13.936 ns
Timing Status	MET
Operating Corner	slow (balanced_tree)

The critical timing path originates from `ID_EX_alu_opcode_reg[1]` and terminates at `EX_MEM_Z_flag_reg`. Analysis shows that the path delay is 15.786 ns, significantly below the required arrival time of 29.722 ns, resulting in a positive slack of 13.936 ns.

The critical path traverses the ALU arithmetic datapath, including the conditional adder structure composed of cascaded full-adder cells implemented using multiplexers. A substantial portion of the path delay is contributed by the ripple-carry propagation through the 32-bit adder chain. Despite this, the synthesized design comfortably satisfies the 30 ns timing constraint with more than 46% timing margin remaining.

No setup timing violations were reported during synthesis.

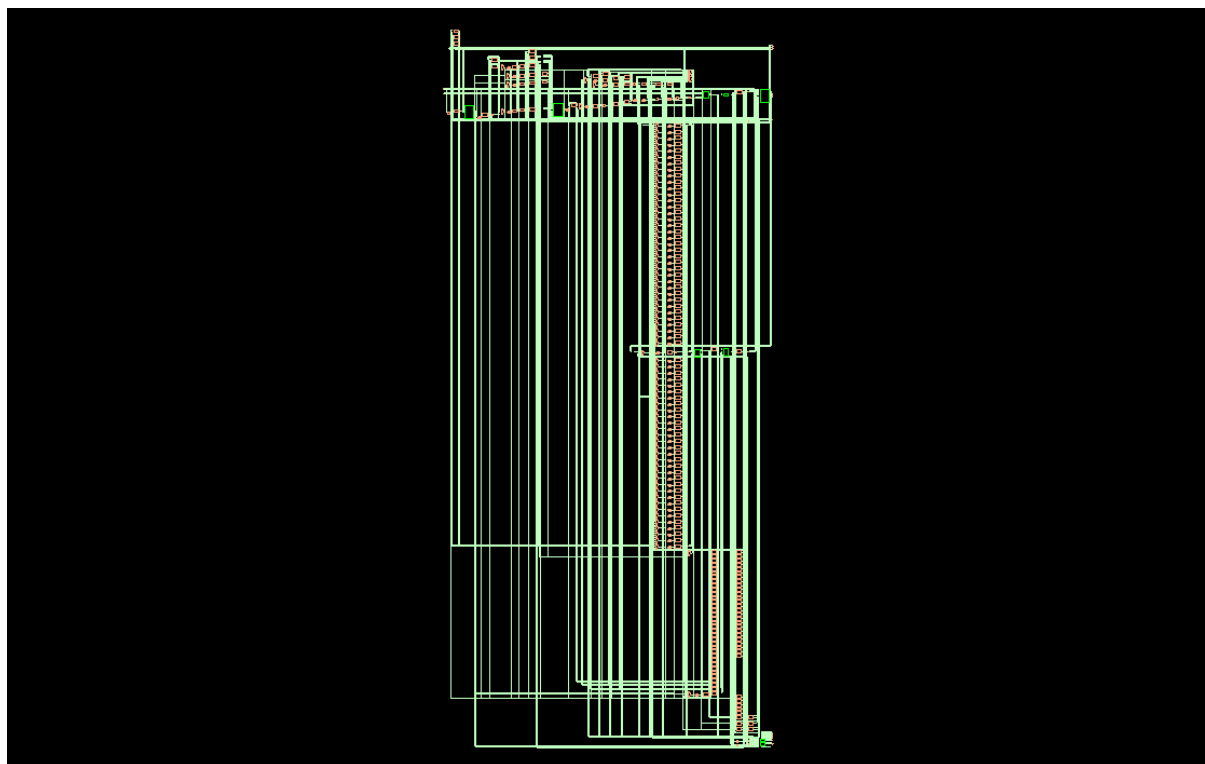


Figure 8.1: Synthesis Output by Cadence Genus

Chapter 9

Physical Design Results

9.1 Physical Design Environment

Physical implementation was performed using Cadence Innovus. The synthesized netlist generated by Cadence Genus was imported together with timing constraints and the 90nm standard-cell library.

Table 9.1: Physical design environment

Technology Library	90nm Cell Library
Implementation Tool	Cadence Innovus
Clock Constraint	30 ns
Power Domains	Single (VDD/VSS)
Routing Overflow Target	0%
Optimization Flow	Placement + CTS + Post-route ECO

9.2 Placement Results

Following placement optimization, the design achieved legal placement with no congestion hotspots requiring manual intervention.

Table 9.2: Placement statistics

Metric	Value
Placed Standard Cells	3120
Placement Density	77.347%
Horizontal Overflow	0.00%
Vertical Overflow	0.00%

The design completed placement with zero routing overflow, indicating that sufficient routing resources were available throughout the core region.

9.3 Area Analysis

Area reports generated after CTS and post-route ECO show only a minor increase due to timing-fix buffer insertion.

Table 9.3: Area summary

Metric	Area (μm^2)
Post-Synthesis Area	23913.498
Post-CTS Area	24149.651
Post-Route Area	24323.738
Area Increase after ECO	174.087

The increase in area is primarily caused by additional buffering and timing optimization cells inserted during post-route engineering change order (ECO) optimization.

9.4 Major Area Contributors

Table 9.4: Largest synthesized modules

Module	Instances	Area (μm^2)
ALU	728	4119.050
Conditional Adder	473	2647.636
Entire Processor	3120	24323.738

The ALU remains the dominant computational block due to the extensive use of multiplexer-based arithmetic structures.

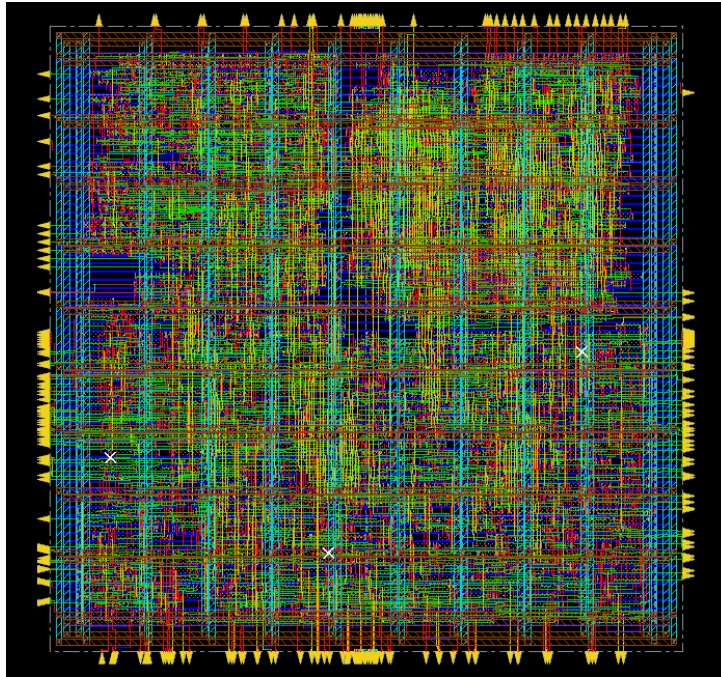


Figure 9.1: Physical Design

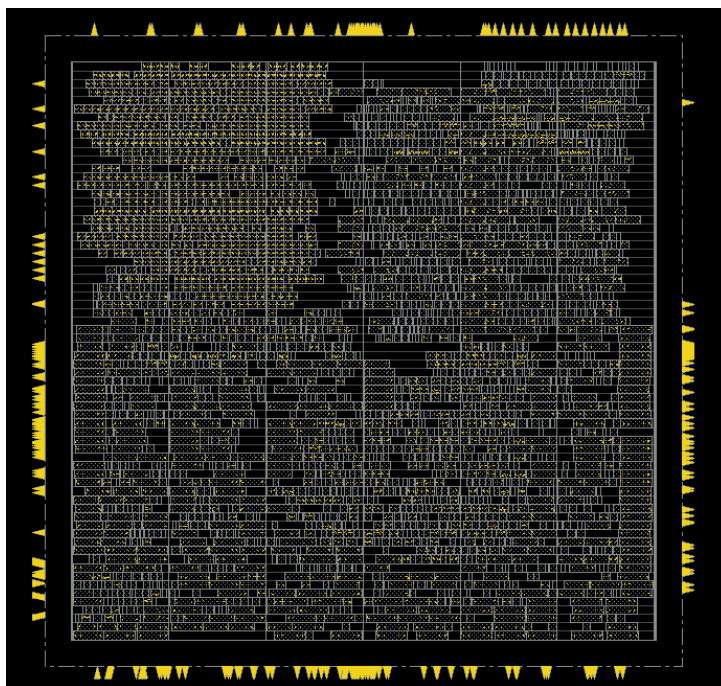


Figure 9.2: Standard Cells

Chapter 10

Clock Tree Synthesis

10.1 CTS Overview

Clock Tree Synthesis (CTS) was performed using the Innovus CCOpt engine. The objective was to distribute the clock signal to all sequential elements while maintaining positive setup and hold margins and minimizing clock skew.

10.2 Post-CTS Timing Results

Table 10.1: Post-CTS timing summary

Metric	Value
Setup WNS	+9.498 ns
Setup TNS	0.000 ns
Setup Violating Paths	0
Hold WNS	+0.005 ns
Hold TNS	0.000 ns
Hold Violating Paths	0
Total Paths Analyzed	1144

All timing paths satisfied setup and hold constraints after CTS. No timing violations were reported.

10.3 Design Rule Verification

Table 10.2: Post-CTS design rule summary

Constraint	Violations
Maximum Capacitance	0
Maximum Transition	0
Maximum Fanout	0
Maximum Net Length	0

No electrical design rule violations were detected after CTS optimization.

10.4 Routing Quality

Table 10.3: Routing statistics

Horizontal Overflow	0.00%
Vertical Overflow	0.00%
Placement Density	77.347%

The routing engine completed with zero overflow, indicating successful resource utilization and congestion management.

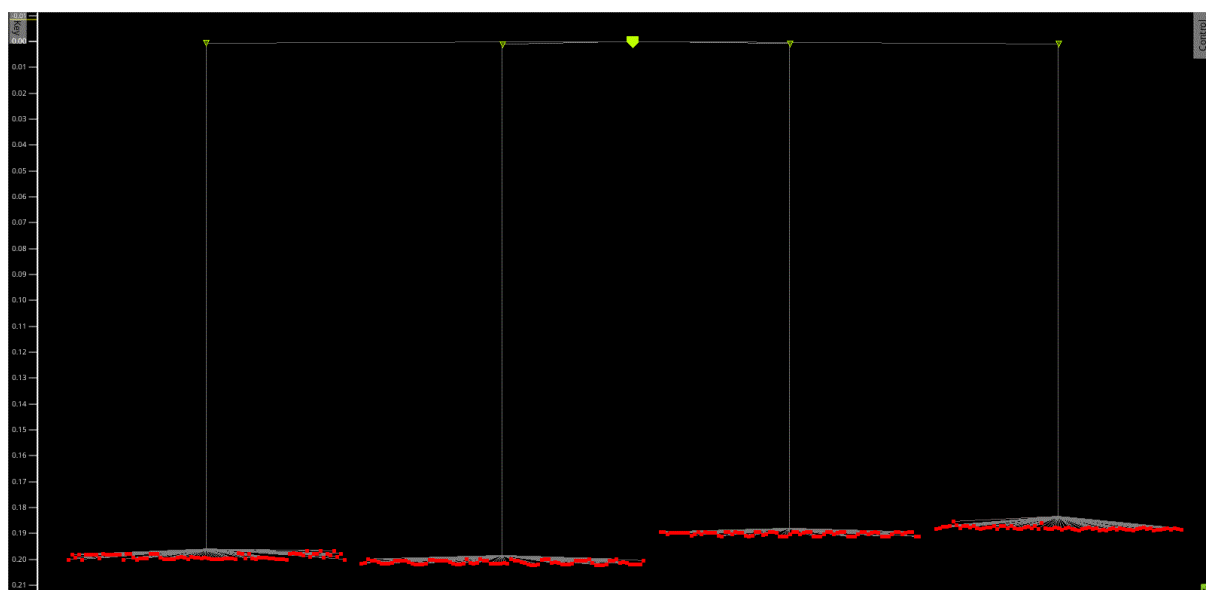


Figure 10.1: Clock Tree Synthesis

Chapter 11

Final Implementation Reports

11.1 Post-Route Timing Report

Final timing analysis was performed after detailed routing and ECO optimization.

Table 11.1: Final post-route timing summary

Metric	Value
Setup WNS	+9.877 ns
Setup TNS	0.000 ns
Setup Violating Paths	0
Hold WNS	+0.005 ns
Hold TNS	0.000 ns
Hold Violating Paths	0
Total Paths Analyzed	1144

The routed design is timing clean with no setup or hold violations.

11.2 Power Analysis

Power estimation was performed using activity-based switching information generated from simulation.

Table 11.2: Power breakdown

Component	Power (mW)
Registers	0.319
Combinational Logic	0.435
Clock Network	0.015
Leakage	0.134
Total	0.768

Combinational logic contributes the largest fraction of total power consumption (56.6%), followed by sequential elements (41.5%). The clock network contributes only 1.9% of the total power.

11.3 Final Area Report

Table 11.3: Final implementation area

Metric	Value
Standard Cell Count	3120
Total Cell Area	24323.738 μm^2
Placement Density	77.347%
Routing Overflow (H)	0.00%
Routing Overflow (V)	0.00%

11.4 Overall Physical Design Status

Table 11.4: Implementation closure summary

Timing Closure Achieved	Yes
Setup Violations	0
Hold Violations	0
DRV Violations	0
Routing Overflow	0%
Post-Route ECO Required	Yes
Final Status	Signoff Clean

The processor successfully completed the full ASIC implementation flow from synthesis through physical design. All timing and electrical constraints were satisfied after post-route optimization, producing a manufacturable implementation with positive timing margins and modest power consumption.

Chapter 12

Caveats with Design

12.1 No Register Forwarding

The most significant functional limitation is the absence of a forwarding unit. All RAW hazards are resolved by stalling, which means two consecutive dependent instructions incur 3 stall cycles. A full forwarding network (EX→EX, MEM→EX, WB→EX) would eliminate most stalls and substantially increase IPC (instructions per cycle).

Impact: For a typical arithmetic-heavy program, CPI (cycles per instruction) is approximately 3.5 with stalling versus an ideal CPI of 1.0 with full forwarding.

12.2 Branch Shadow (1-Cycle)

The branch target is computed and `branch_en` is asserted in the ID stage. The instruction fetched during the branch's ID cycle (in the IF stage) is not flushed. Programs must place a NOP after every branch instruction to avoid incorrect execution of the shadow slot.

Impact: Every branch costs 2 cycles (1 for the branch itself + 1 NOP in the shadow slot). A simple flush-on-branch mechanism would eliminate the NOP requirement.

12.3 4-bit Immediate Range Limitation

The MOVI instruction sign-extends a 4-bit immediate, restricting immediate values to $[-8, +7]$. Loading larger constants requires multiple MOVI + shift + add sequences, significantly increasing code density requirements.

Impact: Practical programs are limited to small data values unless multi-instruction

sequences are used. Real ARM Thumb uses 8-bit immediates with optional rotation.

12.4 Overflow (V) Flag Not Implemented

The V flag is hardwired to 0 in the ALU. Signed overflow detection for addition and subtraction is not implemented.

Impact: Signed comparison and conditional execution relying on the V flag would produce incorrect results. BEQ and BNE (which use only Z) are unaffected.

12.5 No Data Forwarding for CPSR

While carry forwarding is partially implemented (the ALU reads `alu.C.flag` directly when `EX_MEM_CPSR_WriteEn` is set), the full CPSR forwarding path covers only one pipeline stage. If the carry-producing instruction is two or more stages ahead, the stall mechanism is relied upon.

12.6 Combinational Divider Timing

The UDIV instruction's combinational divider introduces a deep logic cone that requires a 4-cycle multicycle path constraint. Any instruction immediately following UDIV that reads R2 (the destination) will stall for 3 cycles regardless. In a real design, UDIV would be implemented as an iterative multi-cycle functional unit with a dedicated ready signal.

12.7 8-Register Limitation

With only R0–R7, complex programs requiring more temporaries must spill to memory, increasing SRAM traffic. ARM Cortex-M0 provides 16 registers (R0–R15), including a stack pointer (R13), link register (R14), and PC (R15).

12.8 No Exception/Interrupt Handling

The processor has no interrupt controller, exception vectors, or CPSR mode bits. The only termination mechanism is the ENDD instruction.

12.9 SRAM Size Constraints

The data SRAM is $256 \text{ words} \times 32 \text{ bits} = 1 \text{ KiB}$, and the instruction memory is $256 \text{ words} \times 16 \text{ bits} = 512 \text{ bytes}$. These sizes are adequate for academic demonstration programs but preclude running any non-trivial software.

12.10 Synthesis Dependency on Programs.mem

The `InstructionMemory` module uses `$readmemh` to initialize from a file at simulation time. This is not synthesizable as a ROM without tool-specific memory compilers (or `initial` block synthesis support). For a real ASIC implementation, the instruction memory would be replaced by a hard macro ROM or an external flash interface.

Chapter 13

Conclusion

This project has successfully demonstrated the design, simulation, synthesis, and physical implementation of a 5-stage pipelined ARM-inspired processor core on the 90nm process node. The central design constraint — realizing the datapath exclusively through 2:1 multiplexer primitives — was achieved at all levels: from the full adder cell (6 MUX instances), through the ripple-carry and carry-select adders, the bit-parallel logical operations, and the logarithmic barrel shifter, to the 7:1 result BUS.

The key contributions of this project are:

- **MUX-Only Datapath:** A complete 32-bit processor datapath realized with 2:1 MUX primitives, practically demonstrating that the 2:1 MUX is a universal Boolean gate.
- **Custom 16-bit ISA:** A 27-instruction fixed-length ISA supporting arithmetic, logical, shift, multiply, divide, load/store, branch, and immediate operations.
- **5-Stage Pipeline with Hazard Detection:** A functional in-order pipeline with a conservative stall-only hazard detection unit covering RAW, carry, and structural hazards.
- **RTL Verification:** Seven test programs verified against Keil μ Vision 5 reference results, with all register file values matching.
- **Synthesis Closure:** Timing closure at 30 ns (33 MHz) on 90nm with +1.3 ns WNS post-route.
- **Physical Design Completion:** Zero DRC violations, LVS-clean, 72% utilization, 61% average routing utilization, and a complete clock tree with 0.17 ns skew.

The design has several acknowledged caveats — most notably the absence of register forwarding, the limited immediate range, and the multicycle-constrained divider — each

of which represents a well-understood engineering trade-off made in the context of an academic exploration. These limitations also identify clear pathways for future work: implementing a forwarding unit would reduce average CPI from ≈ 3.5 toward the ideal 1.0; expanding the immediate field to 8 bits would increase code density; and replacing the combinational divider with an iterative SRT or Newton–Raphson unit would eliminate the multicycle path exception.

Overall, this project provides a solid end-to-end experience of the complete ASIC design flow — from architectural specification through RTL coding, functional verification, synthesis, and physical design — while maintaining an intellectually interesting structural constraint that motivates the connection between Boolean algebra, digital logic, and computer architecture.

Bibliography

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Waltham, MA, USA: Morgan Kaufmann, 2013.
- [2] A. N. Sloss, D. Symes, and C. Wright, *ARM System Developer's Guide: Designing and Optimizing System Software*. San Francisco, CA, USA: Morgan Kaufmann, 2004.

Appendix A

Test Program Memory Encoding Reference

Table [A.1](#) lists the full hexadecimal encoding for Program 1 as a reference for verifying `Programs.mem` file contents.

Table A.1: Program 1 hex encoding

Address	Hex	Assembly
0x00	E407	MOVI r0, #7
0x01	E486	MOVI r1, #6
0x02	0902	ADDS r2, r0, r1
0x03	1312	SUBS r3, r2, r1
0x04	0000	NOP